

ApplyQueue.c 코드 설명

🔖 개요

이 코드는 **원형 큐(Circular Queue)**를 배열로 구현한 예제입니다. **AlignmentQueue.c**의 단점을 개선해, 삭제된 공간을 재활용할 수 있게 만든 구조입니다.

☑ 주요 구조 및 기능 설명

▶ 구조체 정의

```
#define MAX_QUEUE_SIZE 5

typedef int element;
typedef struct {
    element data[MAX_QUEUE_SIZE];
    int front, rear;
} QueueType;
```

- **front**와 **rear**: 삽입/삭제를 추적하는 인덱스
- **MAX_QUEUE_SIZE**: 큐의 전체 크기 (한 칸은 비워둠)

▶ 구현 함수

초기화 및 상태 확인

```
void init_queue(QueueType *q);
int is_empty(QueueType *q);
int is_full(QueueType *q);
```

- **init_queue**: front = rear = 0 으로 초기화
- **is_empty**: front == rear
- **is_full**: (rear + 1) % MAX == front (한 칸 비워둠)

삽입 (enqueue)

```
void enqueue(QueueType *q, element item) {
    if (is_full(q)) error("Queue is full");
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;
    q->data[q->rear] = item;
}
```

삭제 (dequeue)

```
element dequeue(QueueType *q) {
    if (is_empty(q)) error("Queue is empty");
    q->front = (q->front + 1) % MAX_QUEUE_SIZE;
    return q->data[q->front];
}
```

출력 (queue_print)

```
void queue_print(QueueType *q);
```

- 원형 큐 구조에 따라 순환하면서 요소를 출력함

학습 예시: 작동 순서

```
init_queue(&q);           // front = rear = 0
enqueue(&q, 10);          // rear = 1 → data[1] = 10
enqueue(&q, 20);          // rear = 2 → data[2] = 20
dequeue(&q);              // front = 1 → 반환 = 10
enqueue(&q, 30);          // rear = 3 → data[3] = 30
```

내부 상태 예시

```
Index:    0    1    2    3    4
Data:     [ ] [10] [20] [30] [ ]
Front: →   1
Rear:           → 3
```

⚠ 경고 & 패턴

- `MAX_QUEUE_SIZE` 중 실제 데이터는 최대 `MAX-1`개까지 삽입 가능 (한 칸 비움)
- 포인터는 **모듈로 연산 %**으로 순환
- `is_full` 조건 중요: `(rear + 1) % MAX == front`

평가

- 선형 큐의 공간 낭비 문제 해결
- 포인터만 이동하므로 효율적
- 큐의 대표적인 실제 구현 방식

파일 명

ApplyQueue.c: 원형 큐를 배열 기반으로 구현한 실용적 예시. 실제 임베디드나 실시간 시스템에서도 활용됨.